



Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico Di Milano

Network Data Analysis Laboratory

prof. Francesco Musumeci

Group G

Alberto Ceresoli, Emanuele Puricelli, Mostafa Hashemiyan

{alberto.ceresoli, emanuele1.puricelli, mostafa.hashemiyan}@mail.polimi.it



- **Optical amplifiers** (Erbium-Doped Fiber Amplifier, EDFA) compensate power attenuation in optical fibers and guarantee sufficient power at the receiver, **Gain** is **not** the **same across** the **wavelengths** (complex transfer function)
- **Estimate P_{out}** from P_{in} to choose the best wavelength for transmission
- **Deep Neural Network regression**, with multiple inputs and multiple outputs



- DATASET
- RAW DATA VISUAL
- DATA PROCESSING
- NN MODEL
- DATASET SPLIT
- TRAINING
- PARAMETER TUNING
- COMPARISON REGRESSION
- XAI

Input/output power profiles for a single EDFA

- Different gain settings:
 - Total input power to the EDFA varies in the $[-9; 9]$ dBm range
 - Total output power is 15 dBm
- One power measurement in each of $N=84$ channels
- 16497 entries with different power profiles (subsets of active channels)

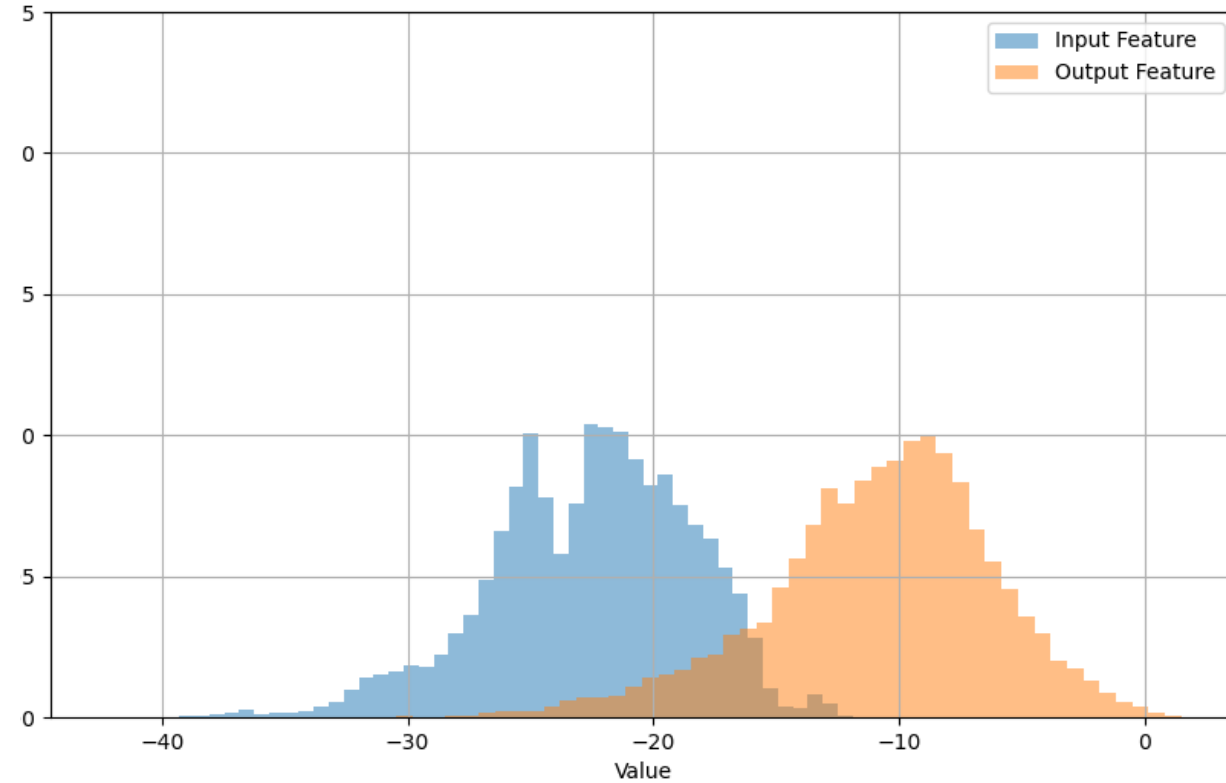
Dataset structure:

Profile Id	Total Power In	Total Power Out	Input profile		Output Profile	
			P. Ch. 1	P. Ch. N	P. Ch. 1	P. Ch. N

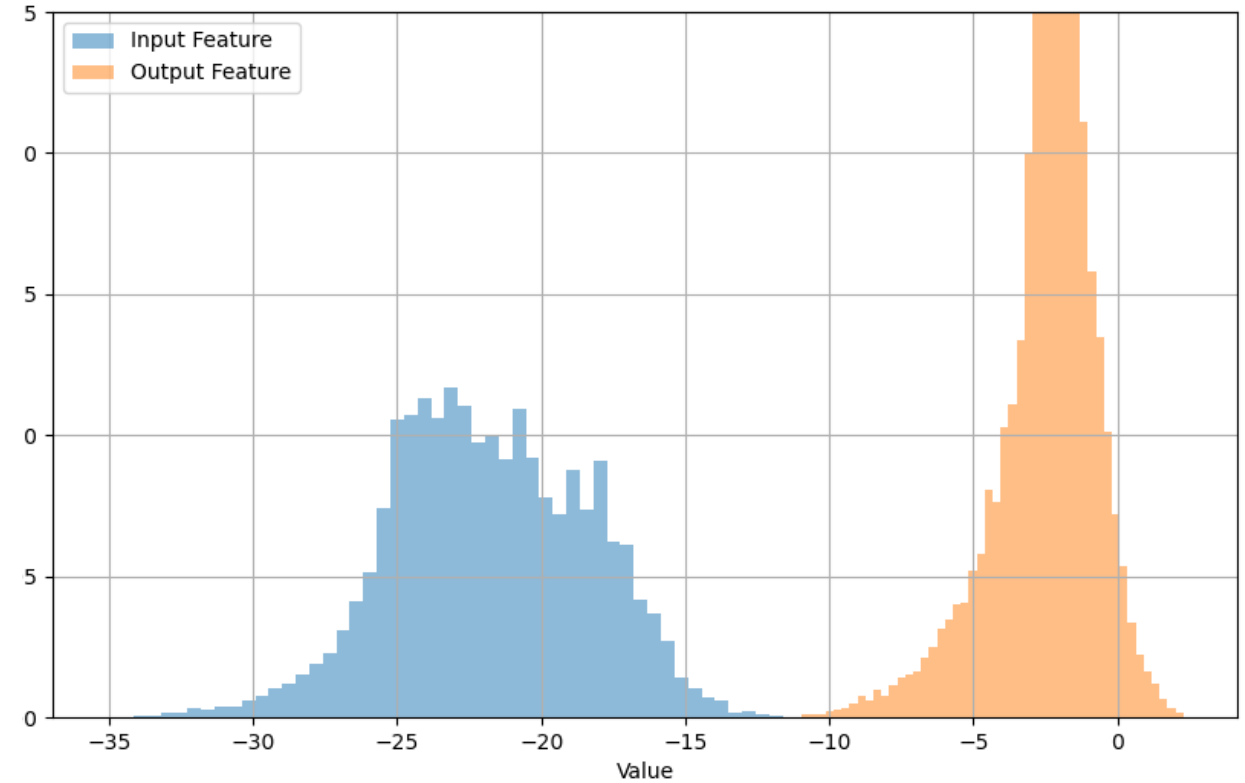
RAW DATA VISUAL



Distribution of inat152970 and outat152970



Distribution of inat156260 and outat156260





```
raw_data.drop(columns=['measuredTotalPout'], inplace=True)
```

```
def extract_features(data, one_every=1):  
    ... freq_chan = 84  
    ... # Extract features from the data  
    ... head = data.columns[:2]  
    ... inPot = data.columns[2:2+freq_chan]  
    ... outPot = data.columns[-freq_chan:]  
    ...  
    ... inPot = inPot[::one_every]  
    ...  
    ... filtered_columns = head.tolist() + inPot.tolist() + outPot.tolist()  
    ... filtered = data[filtered_columns]  
    ...  
    ... freq_chan = len(inPot)  
    ...  
    ... return filtered, freq_chan
```

NULL

or

WRONG values?



CHECK during DATA VISUAL

DATASET SPLIT



```
X = data[data.columns[1:2+freq_chan]]↓  
y = data[data.columns[-84:]]↓
```



```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)↓
```



```
scaler = StandardScaler()↓  
X_train_std = scaler.fit_transform(X_train)↓  
X_test_std = scaler.transform(X_test)↓
```



13197 rows × 85 columns



```
def train_NN(X_tr, y_tr, activ, neur, lyrs):
```

Activation Func, #Neurons, #Layers
parametric for tuning purposes

```
    mae = []  
    mse = []  
    r2 = []
```

MAE, MSE, R2 (, and trainig time)
as evaluation metrics

```
#Homemade kfold k = 5  
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

5-Fold Cross Validation
to increase accuracy and reduce
overfitting

TRAINING



```
for train_index, test_index in kf.split(X_tr, y_tr):  
    X_train_fold, X_test_fold = X_tr[train_index], X_tr[test_index]  
    y_train_fold, y_test_fold = y_tr.iloc[train_index], y_tr.iloc[test_index]  
  
    # Create a neural network regressor  
    model = Sequential()  
    model.add(Input(shape=(X_train_fold.shape[1],)))  
    # Add hidden layers  
    for _ in range(lyrs):  
        model.add(Dense(neur, activation=activ))  
    # Add the output layer  
    model.add(Dense(84)) # 84 outputs, one per channel  
    model.compile(optimizer='adam', loss='mse', metrics=['mse', 'mae', 'r2_score'])  
    print(X_train_fold.shape, y_train_fold.shape)  
    model.fit(X_train_fold, y_train_fold, epochs=15, batch_size=32, validation_split=0.2, verbose=0)  
  
    scores = model.evaluate(X_test_fold, y_test_fold, verbose=0)  
    mse.append(scores[1])  
    mae.append(scores[2])  
    r2.append(scores[3])  
  
    avg_mse = np.mean(mse)  
    avg_mae = np.mean(mae)  
    avg_r2 = np.mean(r2)
```

Optimize Hyperparameters



- Optimize a neural network's hyperparameters (activation function, number of neurons, and layers) for different data sampling rates.
- Evaluate model performance using metrics: MSE (Mean Squared Error), MAE (Mean Absolute Error), and R^2 (coefficient of determination).
- Identify the best-performing model configuration for each one_every sampling value. Store results in CSV files and extract the top-performing setup per configuration.

Tuning parameters



```
for one_every in [10]: #1, 2, 3, 5, 10
    with open(f'hp_opt_metrics/parameters_one_every_{one_every}.csv', 'w') as f:
        f.write(f"Activation,Neurons,Layers,MSE,MAE,R2,Training Time\n")
```

```
print(f'Processing with one_every={one_every}...')
```

```
X_temp, freq_chan = extract_features(raw_data, one_every=one_every)
```

 Extract features

```
X = X_temp[X_temp.columns[1:2+freq_chan]]
```

```
y = X_temp[X_temp.columns[-84:]]
```

Split the dataset

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
# Normalize the data
```

```
scaler = StandardScaler()
```

Standardize the features

```
X_train_std = scaler.fit_transform(X_train)
```

```
X_test_std = scaler.transform(X_test)
```

```
for activ in activation:
```

```
    for neur in neurons:
```

Iterate over hyperparameter

```
        for lyrs in layers:
```

```
            model, training_time, metrics = train_NN(X_train_std, y_train, activ, neur, lyrs)
```

```
            with open(f'hp_opt_metrics/parameters_one_every_{one_every}.csv', 'a') as f:
```

```
                f.write(f"{activ},{neur},{lyrs},{metrics[0]},{metrics[1]},{metrics[2]},{training_time}\n")
```

Which are the best parameters?



```
best_parameters = {}

for one_every in [1, 2, 3, 5, 10]:
    best_parameters[f'one_every_{one_every}'] = {'activation': 'Topo gigio',
                                                  'neurons': -1,
                                                  'layers': -1,
                                                  'mse': 999,
                                                  'mae': 999,
                                                  'r2': -999,
                                                  'training_time': 0
                                                  }

    for line in open(f'hp_opt_metrics/parameters_one_every_{one_every}.csv').readlines()[1:]:
        activ, neur, lyrs, mse, mae, r2, training_time = line.strip().split(',')
        neur = int(neur)
        lyrs = int(lyrs)
        mse = float(mse)
        mae = float(mae)
        r2 = float(r2)
        training_time = float(training_time)

        # Check if the current parameters yield a better score
        if mse < best_parameters[f'one_every_{one_every}']['mse']:
            best_parameters[f'one_every_{one_every}']['activation'] = activ
            best_parameters[f'one_every_{one_every}']['neurons'] = neur
            best_parameters[f'one_every_{one_every}']['layers'] = lyrs
            best_parameters[f'one_every_{one_every}']['mse'] = mse
            best_parameters[f'one_every_{one_every}']['mae'] = mae
            best_parameters[f'one_every_{one_every}']['r2'] = r2
            best_parameters[f'one_every_{one_every}']['training_time'] = training_time
```

- Read the corresponding results CSV file.
- Initialize a dictionary with extreme default values for comparison.
- For each line in the file:
 - Parse metrics and model parameters.
 - Update the best result if the current model has lower MSE.
- Store the best-performing configuration in a dictionary (best_parameters) for future use or reporting.

Sample of finding best combination



Configuration: relu, neurons=16, layers=2

Fold	MSE	MAE	R ²
1	0.1943	0.2980	0.9510
2	0.2095	0.3044	0.9446
3	0.2308	0.3205	0.9437
4	0.2080	0.3075	0.9444
5	0.2309	0.3289	0.9440
Average	0.2147	0.3118	0.9455

Best parameter



one_every	Activation	Neurons	Layers	MSE	MAE	R ²	Training Time (s)
1	relu	128	2	0.04199	0.14574	0.99027	48.49
2	relu	128	2	0.04207	0.14630	0.99017	49.58
3	relu	128	2	0.04201	0.14492	0.99008	49.27
5	relu	128	2	0.04841	0.15454	0.98887	48.81
10	relu	128	3	0.07828	0.19335	0.98237	48.91

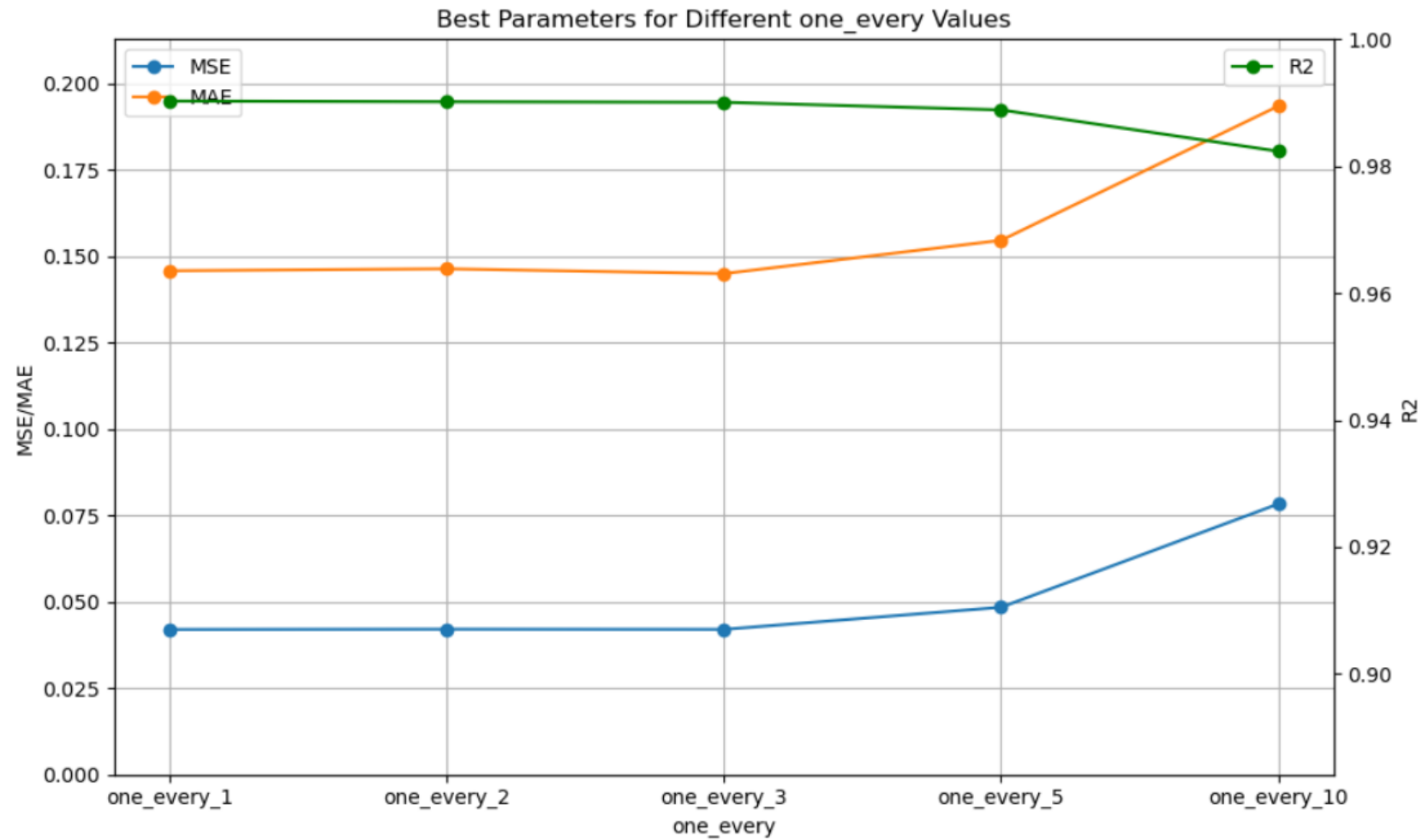
Best parameter



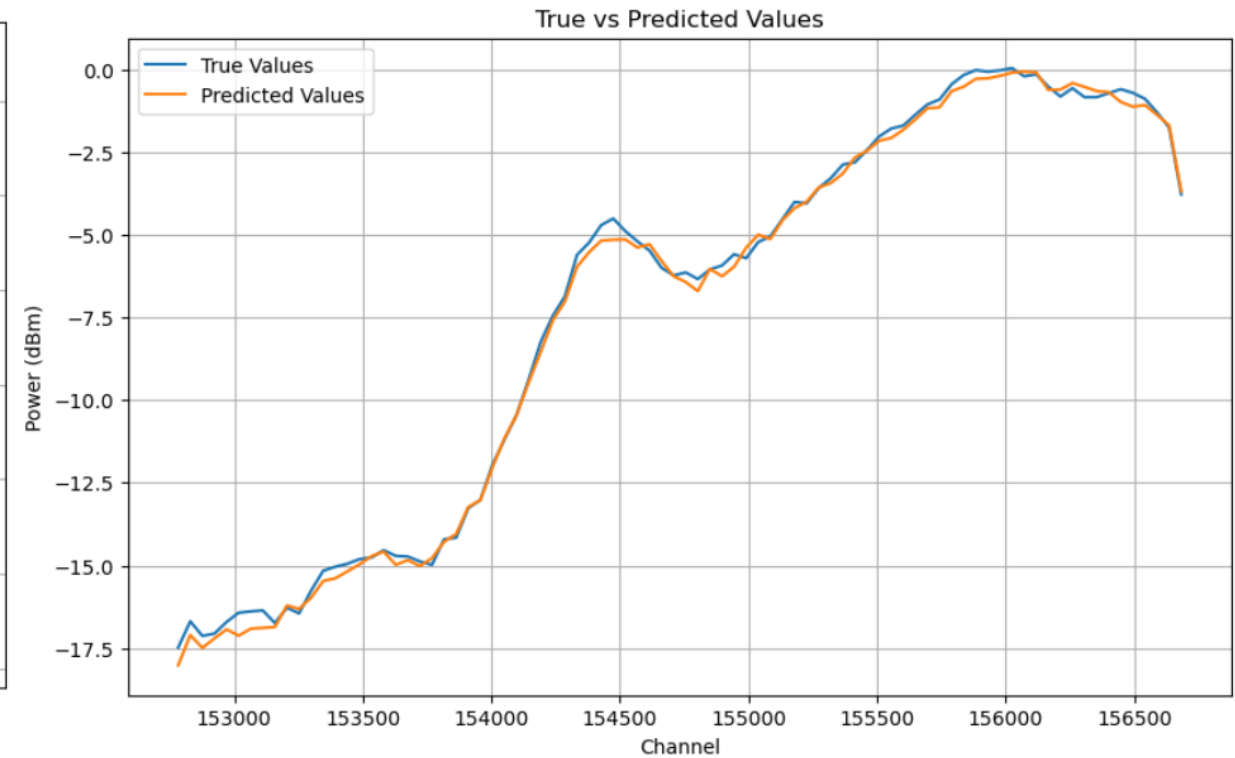
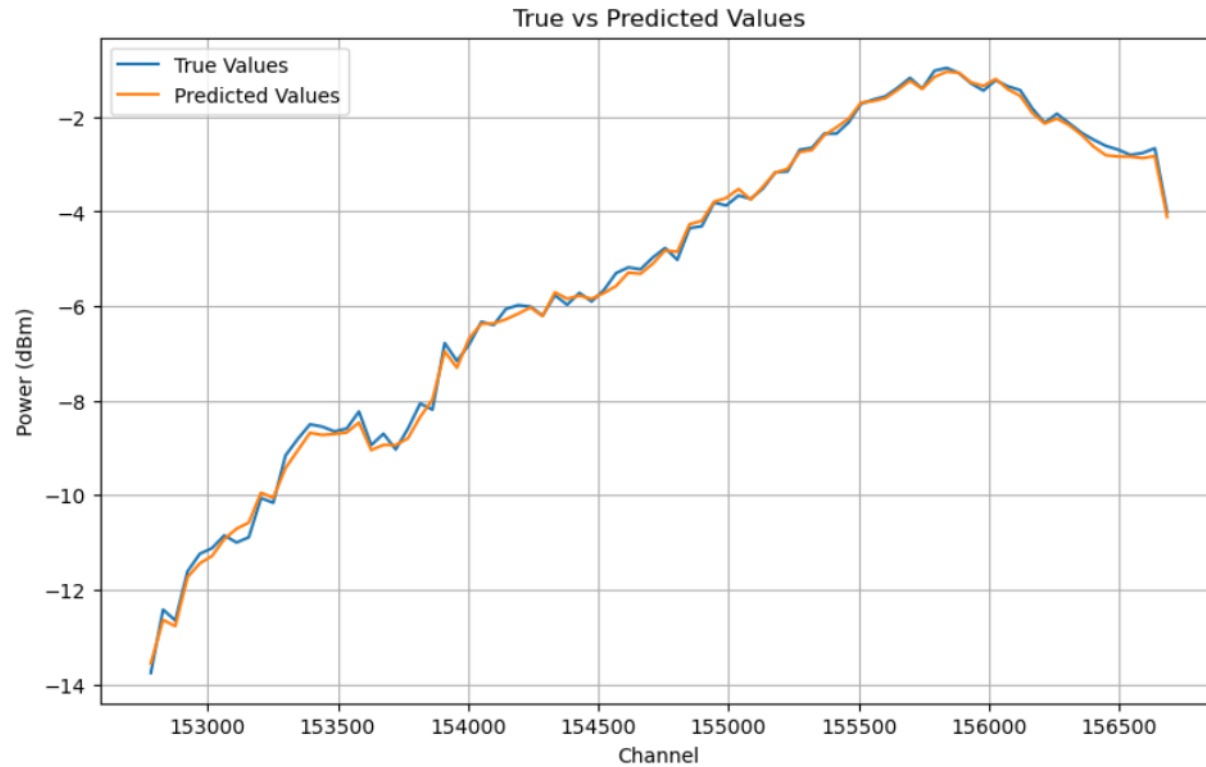
one_every	Activation	Neurons	Layers	MSE	MAE	R ²	Training Time (s)
1	relu	128	2	0.04199	0.14574	0.99027	48.49
2	relu	128	2	0.04207	0.14630	0.99017	49.58
3	relu	128	2	0.04201	0.14492	0.99008	49.27
5	relu	128	2	0.04841	0.15454	0.98887	48.81
10	relu	128	3	0.07828	0.19335	0.98237	48.91

Best "one_every" trade-off --> ~~85~~ 29 input features

Best parameter graph



Training model - RESULTS



Dataset with predicted results



	outat152782	outat152829	outat152876	outat152923	outat152970	outat153017	outat153064	outat153111	outat153158	outat153205	...	outat156260	outat156307	outat156354	outat156401
0	-18.736	-17.176	-16.996	-16.116	-15.856	-15.616	-15.196	-14.746	-14.536	-13.786	...	-0.876	-0.896	-0.916	-0.936
1	-17.462	-15.902	-16.152	-15.302	-14.902	-14.552	-14.142	-13.752	-13.432	-12.792	...	-1.062	-1.142	-1.082	-1.142
2	-15.955	-14.875	-15.225	-14.225	-13.875	-13.475	-13.095	-12.735	-12.505	-11.625	...	-1.285	-1.275	-1.295	-1.375
3	-14.989	-13.799	-13.949	-12.859	-12.579	-12.359	-11.799	-11.439	-11.449	-10.709	...	-1.599	-1.589	-1.619	-1.759
4	-14.292	-12.942	-13.162	-12.062	-11.612	-11.682	-11.432	-11.282	-11.222	-10.582	...	-1.692	-1.842	-1.812	-1.982
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
16492	-11.438	-10.788	-11.028	-9.968	-9.728	-9.418	-8.978	-8.688	-8.548	-7.968	...	-3.938	-4.078	-4.128	-4.208
16493	-11.105	-10.165	-10.365	-9.455	-9.005	-9.335	-8.995	-8.765	-8.675	-8.085	...	-4.145	-4.345	-4.385	-4.565
16494	-10.572	-9.582	-9.842	-9.172	-8.942	-8.572	-8.072	-7.802	-7.682	-7.132	...	-4.142	-4.492	-4.612	-4.792
16495	-9.912	-8.852	-9.602	-8.412	-8.132	-7.962	-7.642	-7.362	-7.142	-6.522	...	-4.382	-4.802	-4.802	-5.152
16496	-9.019	-8.269	-8.619	-7.679	-7.429	-7.549	-7.179	-7.079	-6.899	-6.359	...	-4.369	-4.559	-4.769	-5.269

Linear Regression



```
# Define a function to train and evaluate a regression model
def train_and_evaluate(model, X_train, y_train, X_test, y_test):
    start_time = time.time()
    model.fit(X_train, y_train)
    training_time = time.time() - start_time
    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

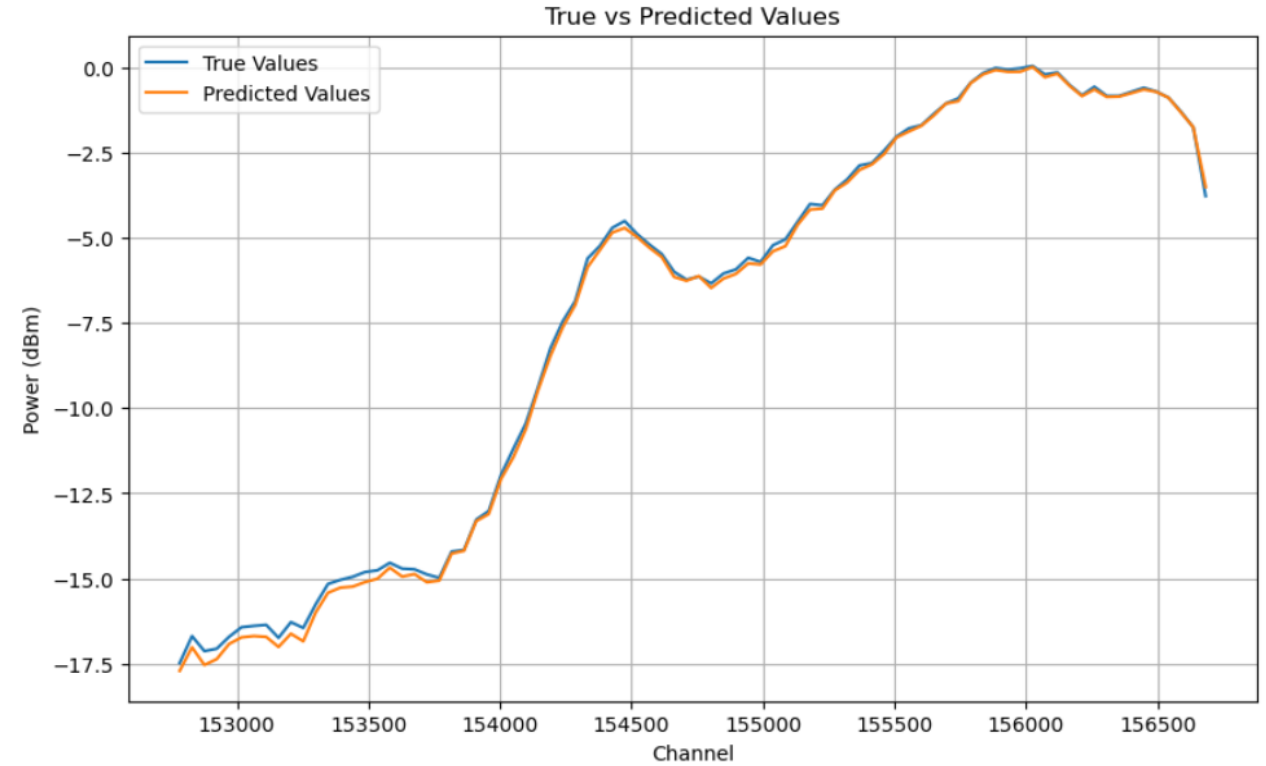
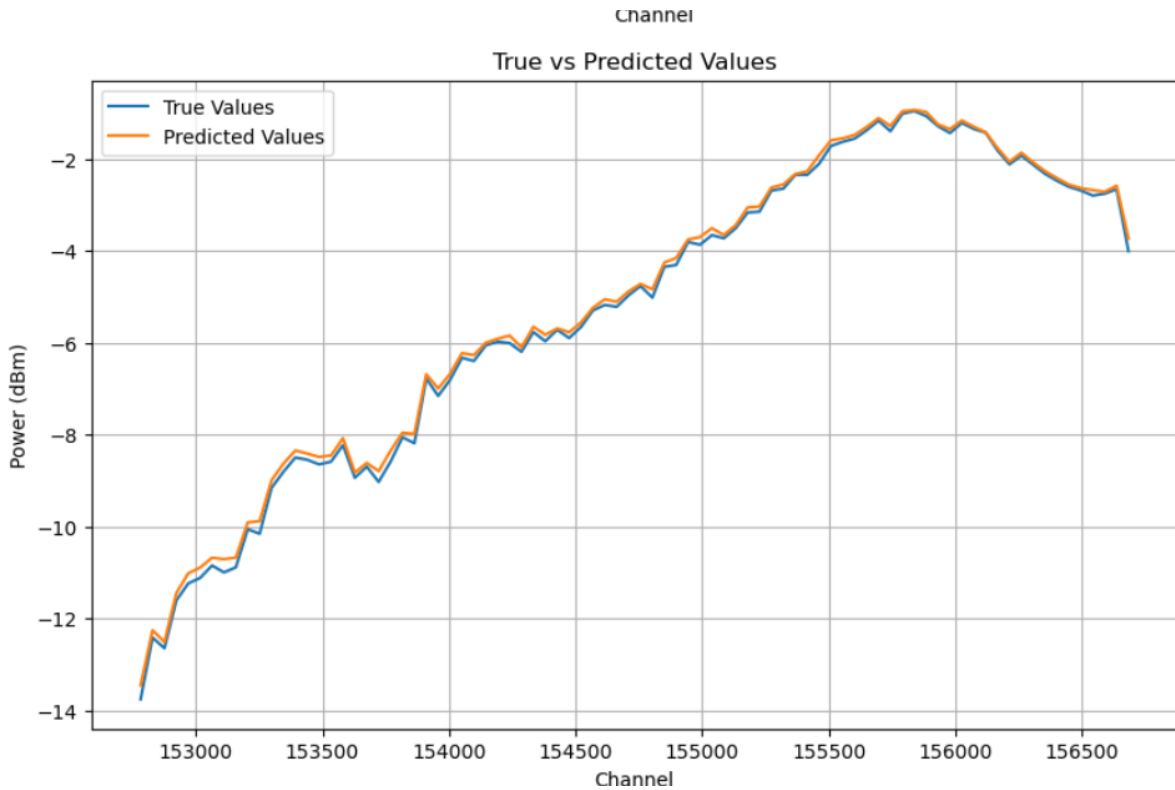
    return mse, mae, r2, training_time

# Initialize models
models = {
    'Linear Regression': LinearRegression(),
}
```

The Metrics of Linear Regression Model

- **Model:** Linear Regression
- **MSE (Mean Squared Error):** 0.032971
- **MAE (Mean Absolute Error):** 0.124289
- **R² (Coefficient of Determination):** 0.992512
- **Training Time:** 0.0363 seconds

Results of simple model LR

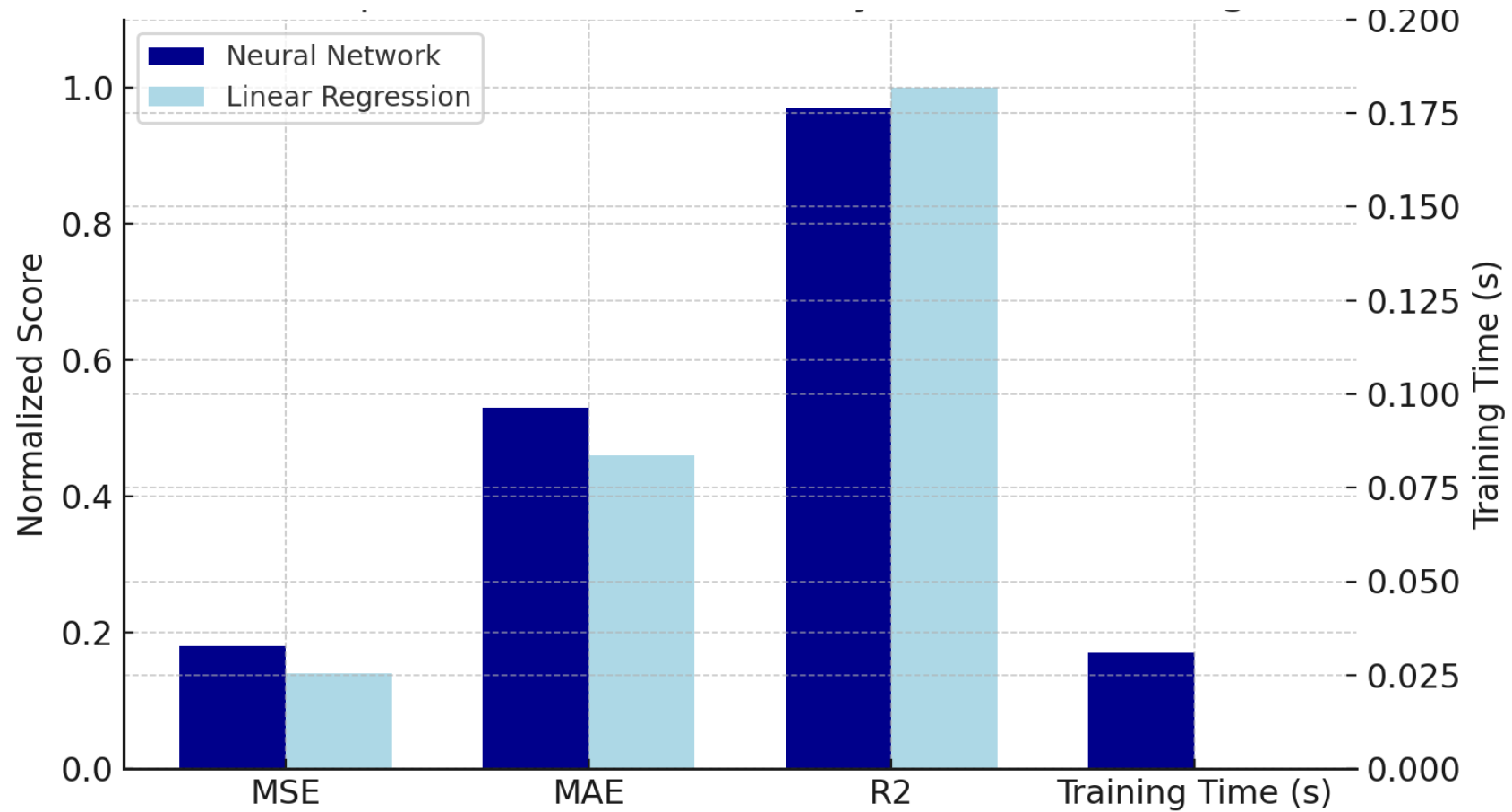


Comparison complex and simple model

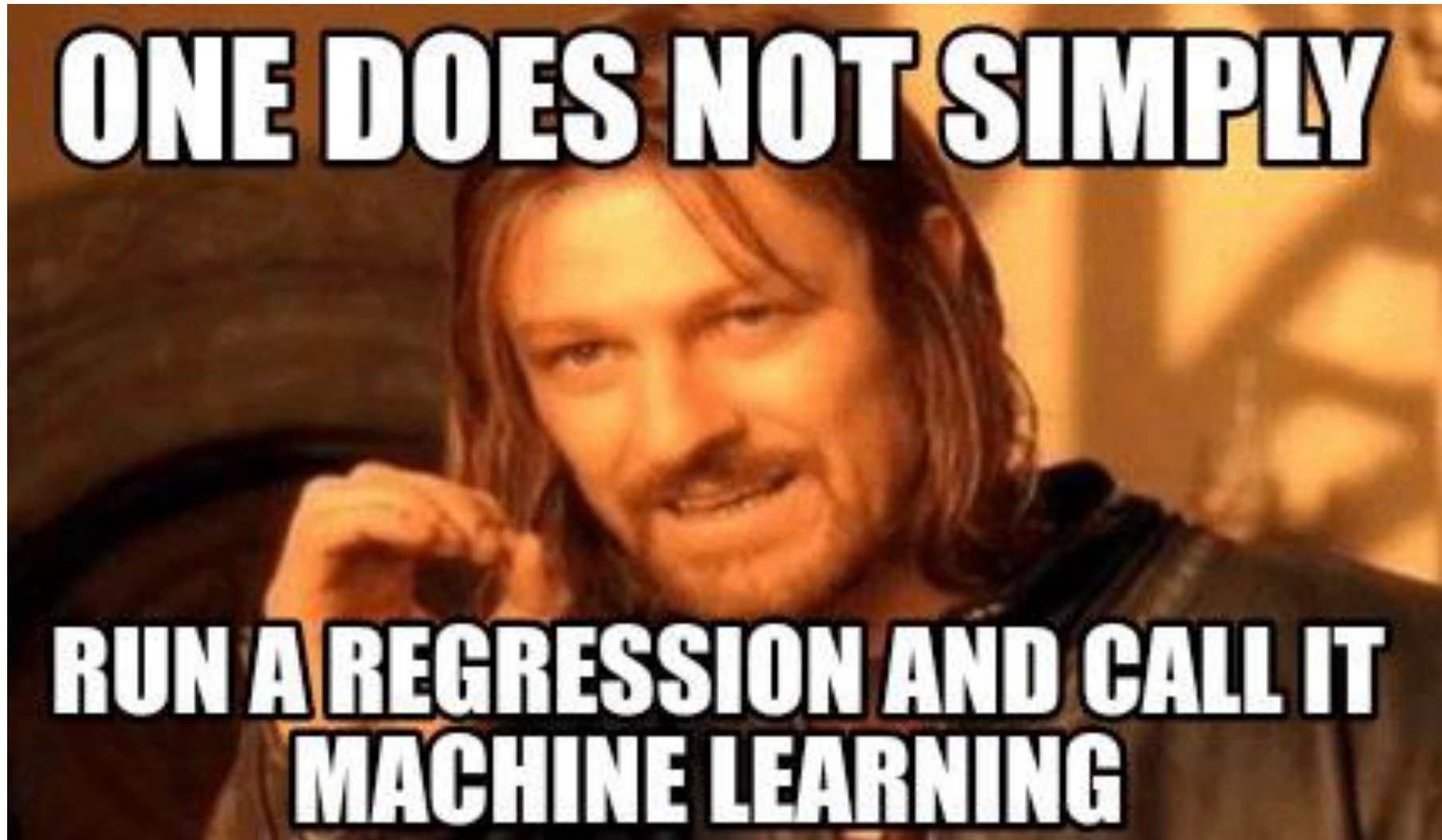


Model	MSE	MAE	R ²	Training Time (s)	Complexity Level	Notes
Neural Network (best: one_every=3)	0.04201	0.14492	0.99008	~49.27 (/5)	☆☆☆ Moderate–High	2 layers, 128 neurons, good accuracy & scalability
Linear Regression	0.03297	0.12429	0.99251	~0.036	☆ Very Low	Fast, interpretable, better performances

Comparison complex and simple model



WAS IT TOO GOOD?



Advanced task : eXplainable AI (XAI)



```
import shap↓
↓
● # Prepare your background and test datasets↓
background = X_train_std[:1000]↓
X_sample = X_test[:100]↓
↓
# Define a prediction function↓
def predict_fn(data):↓
    ... return model.predict(data)↓
↓
# Create the SHAP explainer with permutation algorithm↓
explainer = shap.Explainer(predict_fn, background, algorithm='permutation')↓
↓
# Compute SHAP values↓
shap_values = explainer(X_sample)↓
↓
# Visualize the SHAP values for the first output↓
shap.plots.beeswarm(shap_values[:, :, 0])
```


Advanced task : eXplainable AI (XAI)



```
import shap
↓
# Prepare your background and test datasets
background = X_train_std[:1000]
X_sample = X_test[:100]
↓
# Define a prediction function
def predict_fn(data):
    ... return model.predict(data)
↓
# Create the SHAP explainer with permutation algorithm
explainer = shap.Explainer(predict_fn, background, algorithm='permutation')
↓
# Compute SHAP values
shap_values = explainer(X_sample)
↓
# Visualize the SHAP values for the first output
shap.plots.beeswarm(shap_values[:, :, 0])
```

- Select a subset of dataset (reduce time complexity)

Advanced task : eXplainable AI (XAI)



```
import shap
↓
# Prepare your background and test datasets
background = X_train_std[:1000]
X_sample = X_test[:100]
↓
# Define a prediction function
def predict_fn(data):
    ... return model.predict(data)
↓
# Create the SHAP explainer with permutation algorithm
explainer = shap.Explainer(predict_fn, background, algorithm='permutation')
↓
# Compute SHAP values
shap_values = explainer(X_sample)
↓
# Visualize the SHAP values for the first output
shap.plots.beeswarm(shap_values[:, :, 0])
```

- Select a subset of dataset (reduce time complexity)
- General Explainer for more flexible usage

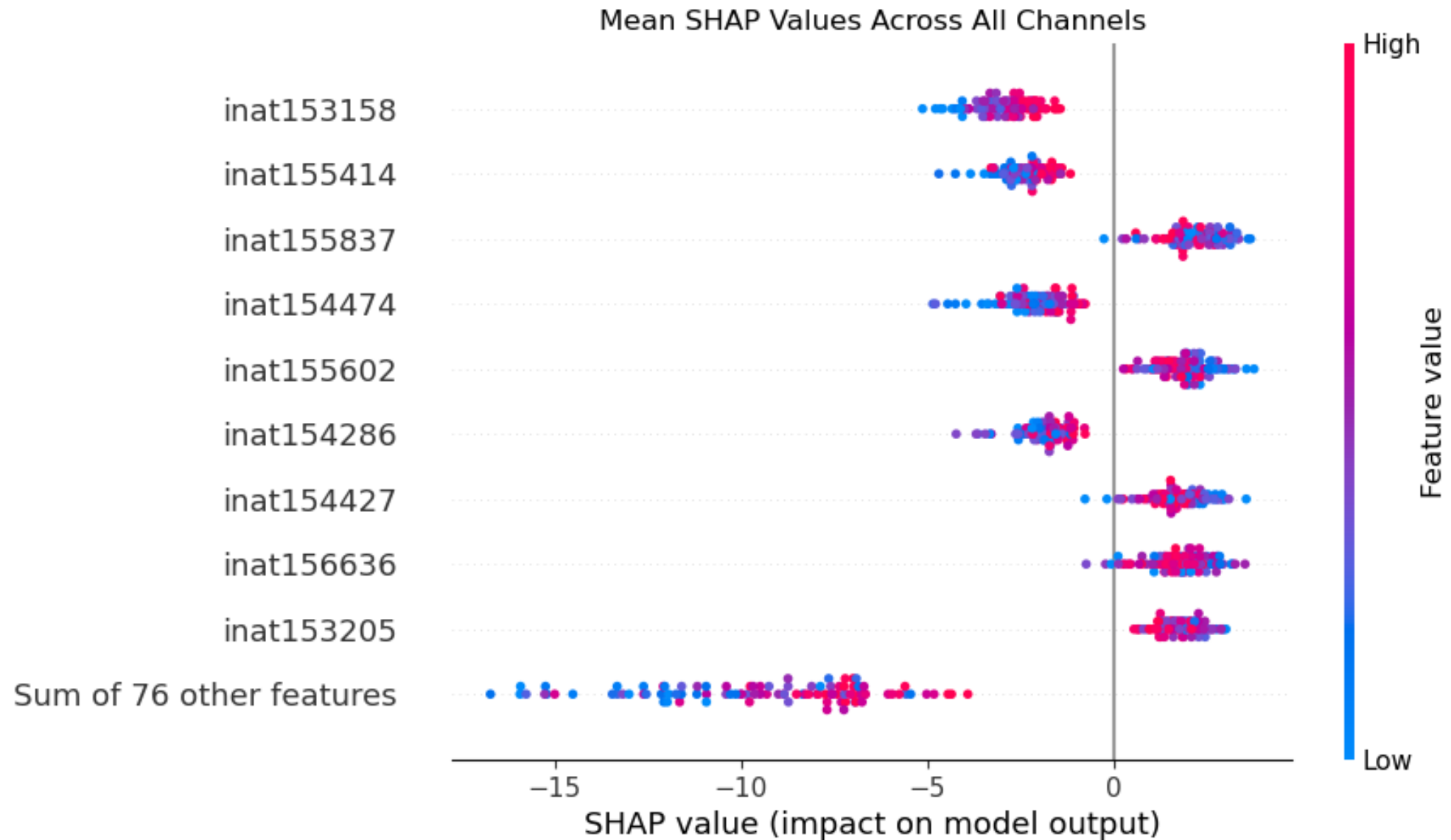
Advanced task : eXplainable AI (XAI)



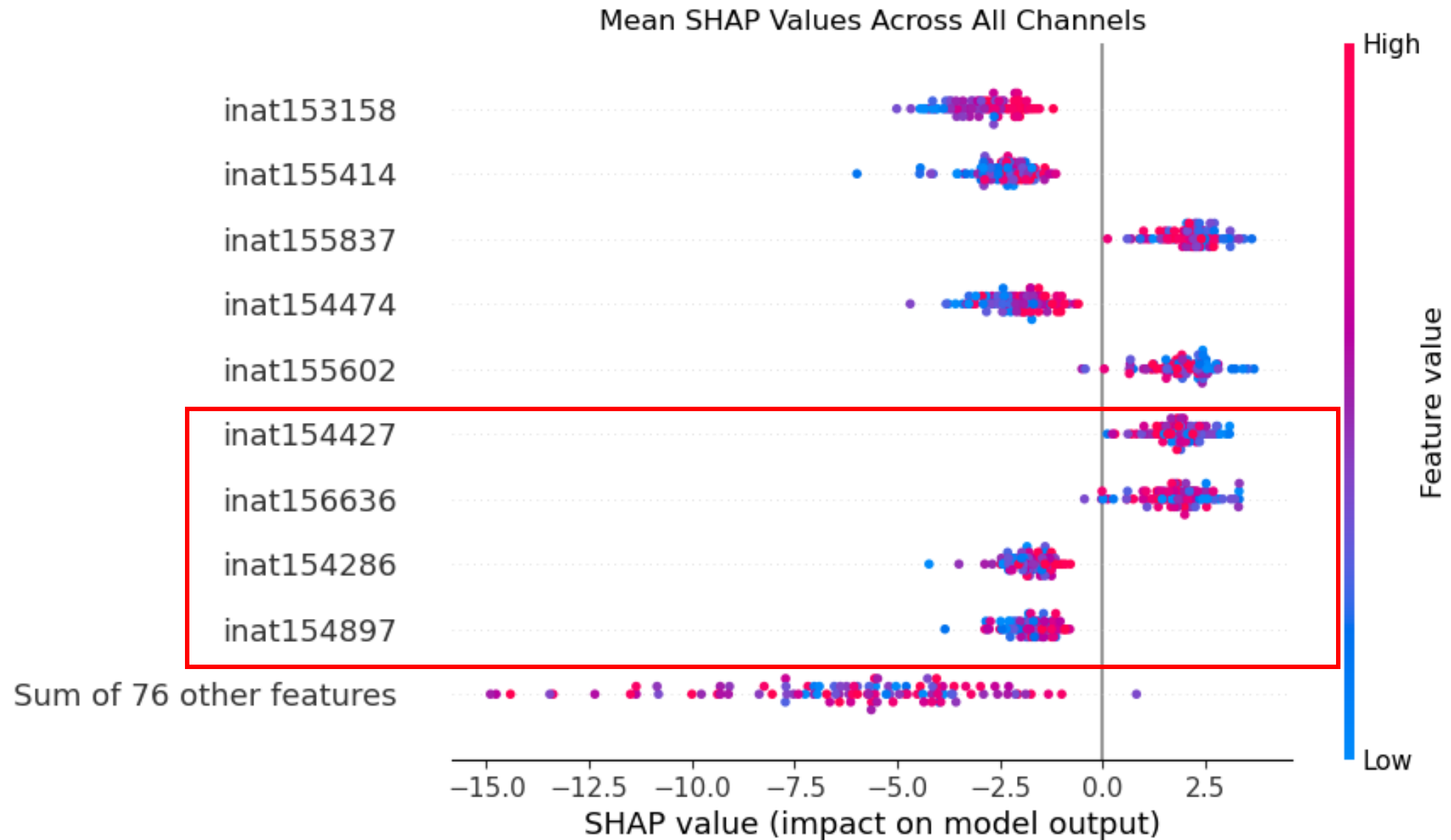
```
import shap
↓
# Prepare your background and test datasets
background = X_train_std[:1000]
X_sample = X_test[:100]
↓
# Define a prediction function
def predict_fn(data):
    ... return model.predict(data)
↓
# Create the SHAP explainer with permutation algorithm
explainer = shap.Explainer(predict_fn, background, algorithm='permutation')
↓
# Compute SHAP values
shap_values = explainer(X_sample)
↓
# Visualize the SHAP values for the first output
shap.plots.beeswarm(shap_values[:, :, 0])
```

- Select a subset of dataset (reduce time complexity)
- General Explainer for more flexible usage
- Permutation algorithm for model agnostic analysis

Advanced task : eXplainable AI (XAI)



Advanced task : eXplainable AI (XAI)

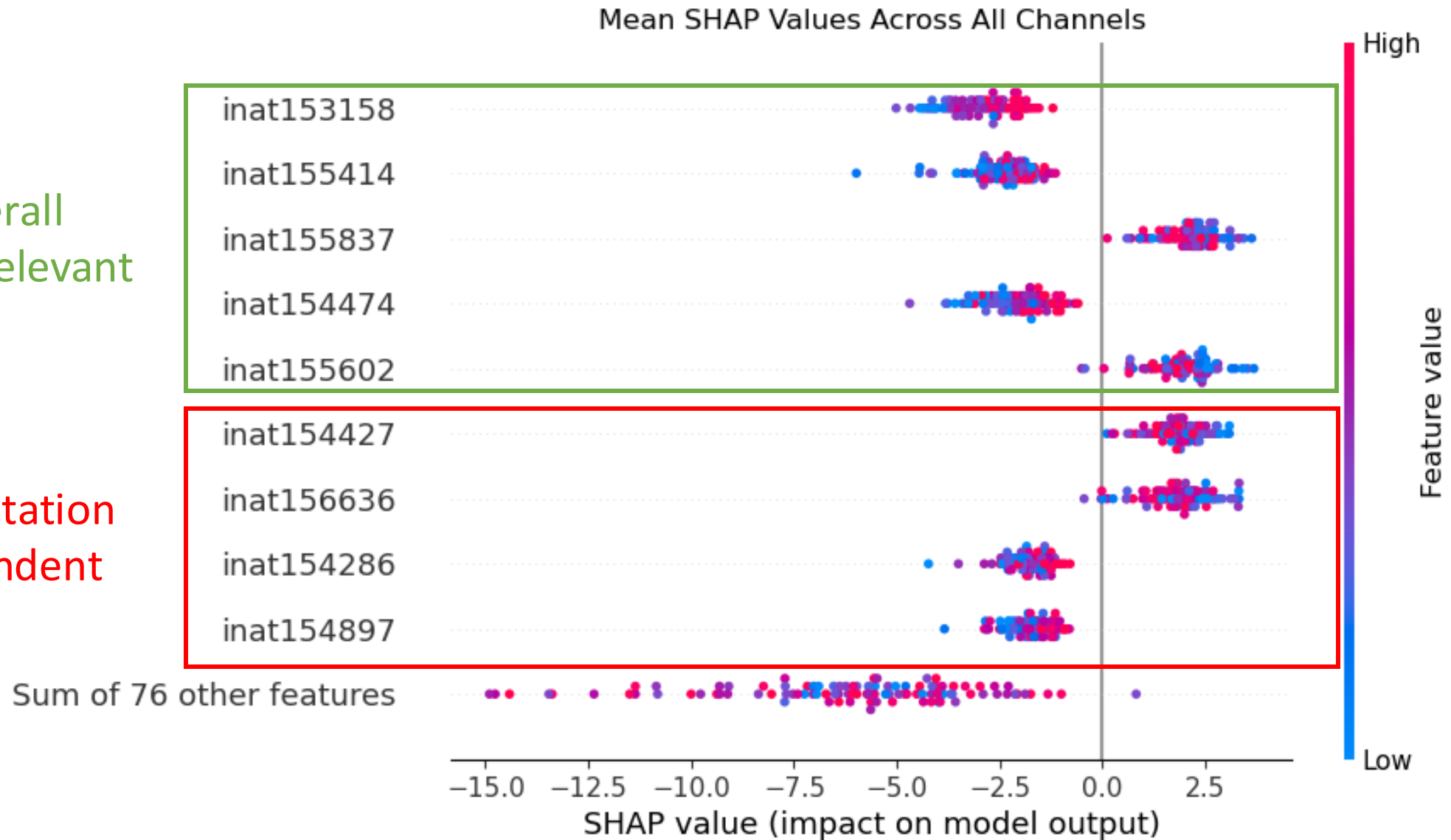


Advanced task : eXplainable AI (XAI)



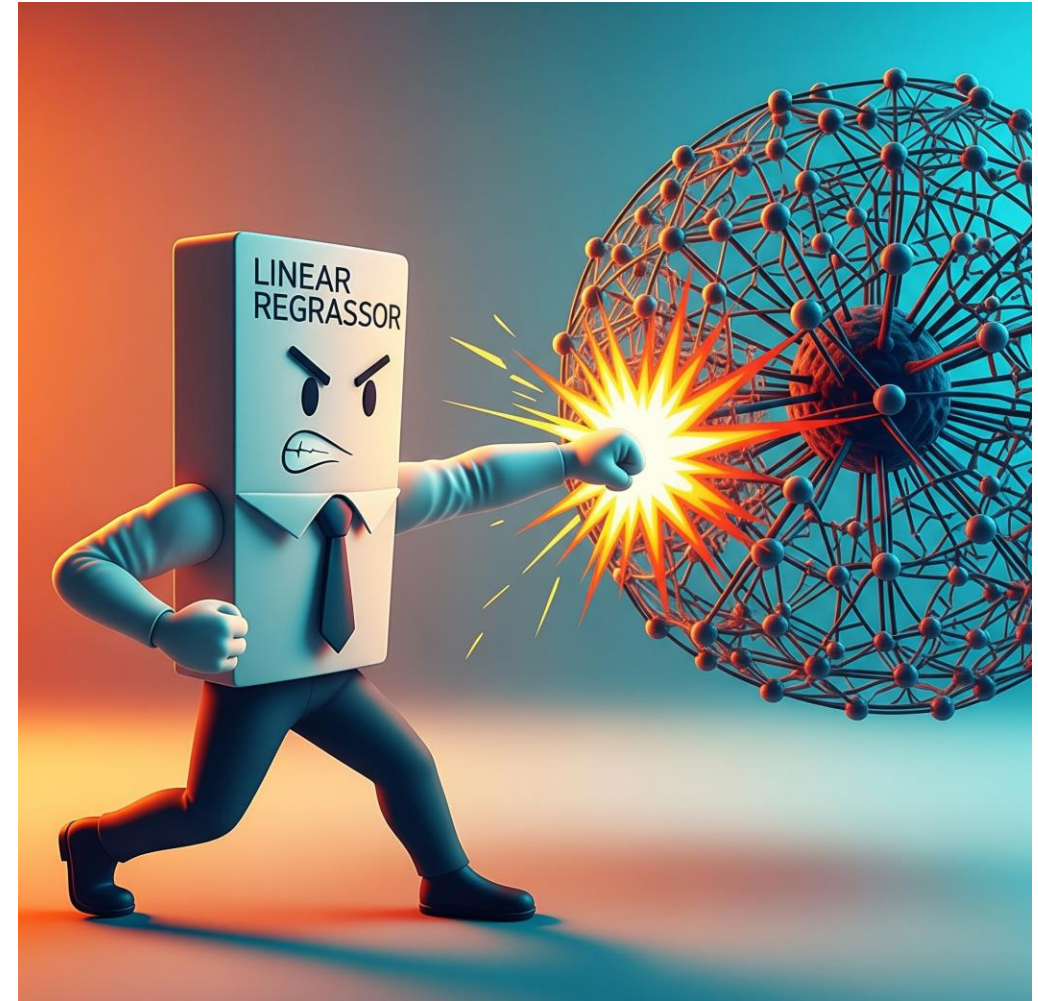
Overall
Most Relevant

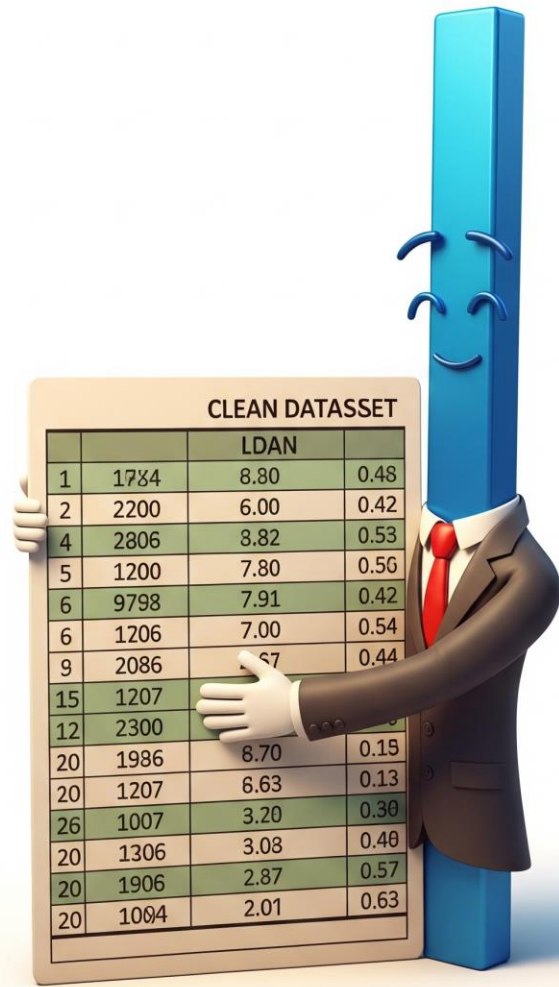
Permutation
Dependent





- **Neural Network** offered a strong balance between accuracy and training time, performing well across all metrics with flexibility for hyperparameter tuning.
- **Linear Regression** slightly outperformed the Neural Network in R^2 and had the fastest training time, making it ideal for quick, interpretable solutions.





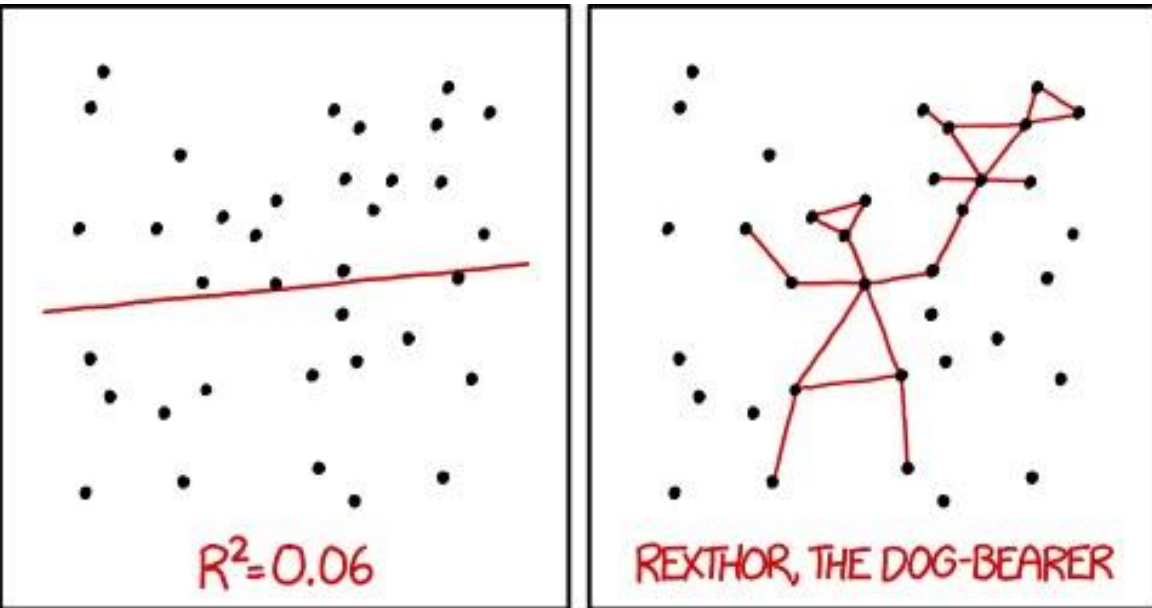
- The **superior performance of Linear Regression** suggests that the dataset is relatively clean and exhibits some degree of linearity between all input channels and output ones (considered one at a time)
- Highlights the **trade-off** between using a simple, efficient model versus a more complex, tunable one.

THIS IS A DRAFT !!!!

This is gonna be
the last slide

Thanks for your attention
and stuff

TOO MANY NICE MEMES



I DON'T TRUST LINEAR REGRESSIONS WHEN IT'S HARDER
TO GUESS THE DIRECTION OF THE CORRELATION FROM THE
SCATTER PLOT THAN TO FIND NEW CONSTELLATIONS ON IT.

